

# Java Collections

## Problem Solving Practice

---

[ArrayList](#) | [HashSet](#) | [HashMap](#) | [Stack](#)

### Notice

### Code Review — Monday

All code for the problems in this document will be reviewed on **Monday** directly from your **GitHub accounts**. Make sure every solution is committed and pushed before Monday morning. Incomplete or missing submissions will be noted.

# Contents

---

## Section 1 — ArrayList

1. Remove Consecutive Duplicates
2. Rotate ArrayList by K Positions
3. Leaders in an Array
4. Move Zeros to End
5. Find Pair With Target Sum

## Section 2 — HashSet

6. Longest Consecutive Sequence
7. Find First Repeated Element
8. Happy Number
9. Common Unique Elements
10. Missing Positive Integer

## Section 3 — HashMap

11. Two Sum
12. Group Anagrams
13. Character Frequency Sort
14. Top K Frequent Elements
15. Find Duplicate File Names

## Section 4 — Stack

16. Valid Parentheses
17. Next Greater Element
18. Remove Adjacent Duplicates
19. Decode String
20. Stock Span Problem

---

*Note: Solutions are not included in this document. Write and test all implementations independently.*

## Section 1 — ArrayList

### 1. Remove Consecutive Duplicates

Given an ArrayList of integers, remove all consecutive duplicate values while keeping the order of first occurrences. Non-consecutive duplicates are kept.

**Input:**

[1, 1, 2, 2, 2, 3, 1, 1, 4]

**Output:**

[1, 2, 3, 1, 4]

**Constraints:**

- $1 \leq \text{list.size()} \leq 10^5$
- Elements can be any integer
- No extra data structures

**Method Signature:**

```
public ArrayList<Integer> removeConsecutiveDuplicates(  
    ArrayList<Integer> list)
```

**Hint:**

- Compare each element with only its immediate predecessor.
- Build result in-place or into a new list while iterating.

### 2. Rotate ArrayList by K Positions

Given an ArrayList of integers and a non-negative integer k, rotate the list to the right by k positions. If k exceeds the list size, apply modulo.

**Input:**

list = [1, 2, 3, 4, 5], k = 2

**Output:**

[4, 5, 1, 2, 3]

**Constraints:**

- $0 \leq k \leq 10^4$
- $1 \leq \text{list.size()} \leq 10^5$
- Return a new list or modify in-place

**Method Signature:**

```
public ArrayList<Integer> rotateRight(  
    ArrayList<Integer> list, int k)
```

**Hint:**

- Normalise k using modulo before rotating.
- `subList()` can simplify slicing the list.

### 3. Leaders in an Array

An element is a leader if it is strictly greater than all elements to its right. The rightmost element is always a leader. Return all leaders in left-to-right order.

**Input:**

[16, 17, 4, 3, 5, 2]

**Output:**

[17, 5, 2]

**Constraints:**

- $1 \leq \text{list.size()} \leq 10^5$
- All elements are distinct
- Return in original order

**Hint:**

- *Traverse from right to left, maintaining a running maximum.*
- *An element is a leader if it exceeds the current right-side max.*

**Method Signature:**

```
public ArrayList<Integer> findLeaders(  
    ArrayList<Integer> list)
```

## 4. Move Zeros to End

Given an ArrayList of integers, move all zero values to the end while maintaining the relative order of all non-zero elements. Minimise write operations.

**Input:**

[0, 1, 0, 3, 12]

**Output:**

[1, 3, 12, 0, 0]

**Constraints:**

- $1 \leq \text{list.size()} \leq 10^5$
- Relative order of non-zero elements must not change
- Modify in-place

**Hint:**

- *Use a write pointer to track the next position for a non-zero value.*
- *After placing all non-zeros, fill remaining indices with zero.*

**Method Signature:**

```
public void moveZerosToEnd(ArrayList<Integer> list)
```

## 5. Find Pair With Target Sum

Given an ArrayList of integers and a target integer, return true if any two distinct elements sum to the target value.

**Input:**

list = [2, 7, 11, 15], target = 9

**Output:**

true

**Constraints:**

- $2 \leq \text{list.size()} \leq 10^5$
- Each element may be used at most once
- $-10^9 \leq \text{element}, \text{target} \leq 10^9$

**Hint:**

- *For each element  $x$ , check if  $(\text{target} - x)$  has been seen already.*
- *A HashSet answers existence queries in  $O(1)$ .*

**Method Signature:**

```
public boolean hasPairWithSum(  
    ArrayList<Integer> list, int target)
```

## Section 2 — HashSet

### 6. Longest Consecutive Sequence

Given an unsorted array of integers, find the length of the longest sequence of consecutive integers. Elements do not need to be contiguous in the input.

**Input:**

```
[100, 4, 200, 1, 3, 2]
```

**Output:**

```
4 // sequence: 1, 2, 3, 4
```

**Constraints:**

- $0 \leq \text{nums.length} \leq 10^5$
- Each number appears at most once
- $O(n)$  time expected

**Method Signature:**

```
public int longestConsecutive(int[] nums)
```

**Hint:**

- Load all elements into a HashSet, then look for sequence starting points.
- A number is a start point only if  $(\text{num} - 1)$  is absent from the set.

### 7. Find First Repeated Element

Given an array of integers, find and return the first element that appears more than once during a left-to-right traversal. Return -1 if no element repeats.

**Input:**

```
[2, 5, 1, 2, 3, 5]
```

**Output:**

```
2
```

**Constraints:**

- $1 \leq \text{nums.length} \leq 10^5$
- Elements can be any integer
- Return the first duplicate by second-occurrence position

**Method Signature:**

```
public int firstRepeated(int[] nums)
```

**Hint:**

- Maintain a HashSet of seen elements; the first hit is the answer.
- Iterate strictly left to right — no sorting.

### 8. Happy Number

Starting from any positive integer, repeatedly replace the number with the sum of the squares of its digits. Return true if the process eventually reaches 1, false if it enters a cycle.

**Input:**

```
19
1^2 + 9^2 = 82
8^2 + 2^2 = 68 ... (eventually reaches 1)
```

**Output:**

```
true
```

**Constraints:**

- $1 \leq n \leq 2^{31} - 1$
- Cycle detection required for non-happy numbers

**Method Signature:**

```
public boolean isHappy(int n)
```

**Hint:**

- Store previously seen sums in a *HashSet* to detect cycles.
- If  $sum == 1$ , it's happy; if already in the set, it's not.

## 9. Common Unique Elements

Given two integer arrays that may contain duplicates, return a list of elements that appear in both arrays. Each element in the result must appear only once.

**Input:**

```
arr1 = [1, 2, 2, 3]
arr2 = [2, 2, 4]
```

**Output:**

```
[2]
```

**Constraints:**

- $0 \leq arr1.length, arr2.length \leq 10^5$
- Result may be in any order
- No duplicate values in output

**Method Signature:**

```
public List<Integer> intersection(
    int[] arr1, int[] arr2)
```

**Hint:**

- Convert one array to a *HashSet*, then check each element of the other.
- Collect matches into a second *HashSet* to avoid duplicates in the result.

## 10. Missing Positive Integer

Given an unsorted integer array, find the smallest missing positive integer. Negative numbers and zero are irrelevant. Do not rely on sorting.

**Input:**

```
[3, 4, -1, 1]
```

**Output:**

```
2
```

**Constraints:**

- $1 \leq nums.length \leq 10^5$
- Answer is always  $\geq 1$
- $O(n)$  time expected

**Method Signature:**

```
public int firstMissingPositive(int[] nums)
```

**Hint:**

- Load all positive integers into a *HashSet*.
- Starting from 1, return the first integer not present in the set.



## Section 3 — HashMap

### 11. Two Sum

Given an array of integers and a target value, return the indices of the two numbers that add up to the target. Exactly one solution exists; the same element may not be used twice.

**Input:**

```
nums = [2, 7, 11, 15], target = 9
```

**Output:**

```
[0, 1] // 2 + 7 = 9
```

**Constraints:**

- $2 \leq \text{nums.length} \leq 10^4$
- Exactly one valid pair exists
- Return indices in any order

**Method Signature:**

```
public int[] twoSum(int[] nums, int target)
```

**Hint:**

- Store each element's value and index in a HashMap as you iterate.
- For each element, check if  $(\text{target} - \text{element})$  is already in the map.

### 12. Group Anagrams

Given an array of strings, group all anagrams together and return a list of groups. Two strings are anagrams if one can be formed by rearranging the letters of the other.

**Input:**

```
["eat", "tea", "tan", "ate", "nat", "bat"]
```

**Output:**

```
[["eat","tea","ate"], ["tan","nat"], ["bat"]]
```

**Constraints:**

- $1 \leq \text{strs.length} \leq 10^4$
- All strings are lowercase English letters
- $0 \leq \text{strs}[i].\text{length} \leq 100$

**Method Signature:**

```
public List<List<String>> groupAnagrams(String[] strs)
```

**Hint:**

- Sort each string alphabetically to get a canonical key.
- Use the sorted form as a HashMap key mapping to a list of original strings.

### 13. Character Frequency Sort

Given a string, sort its characters in descending order of frequency. If two characters have the same frequency, their relative order does not matter.

**Input:**

```
"tree"
```

**Output:**

```
"eert" // e appears twice, r and t once
```

**Constraints:**

- $1 \leq s.length() \leq 5 * 10^5$
- May contain letters, digits, and symbols
- Multiple valid outputs accepted for equal frequencies

**Hint:**

- Build a frequency map with *HashMap*.
- Sort entries by frequency descending, then reconstruct the string.

**Method Signature:**

```
public String frequencySort(String s)
```

## 14. Top K Frequent Elements

Given an integer array and a positive integer  $k$ , return the  $k$  elements that appear most frequently. If two elements have the same frequency, either may be chosen.

**Input:**

```
nums = [1, 1, 1, 2, 2, 3], k = 2
```

**Output:**

```
[1, 2]
```

**Constraints:**

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq k \leq$  number of unique elements
- $-10^4 \leq \text{nums}[i] \leq 10^4$

**Hint:**

- Build a frequency map, then extract the top  $k$  by frequency.
- A min-heap of size  $k$  or bucket sort by frequency both work.

**Method Signature:**

```
public int[] topKFrequent(int[] nums, int k)
```

## 15. Find Duplicate File Names

Given a list of file name strings that may contain duplicates, return all file names that appear more than once. Each duplicate should appear exactly once in the result.

**Input:**

```
["resume.pdf", "photo.png", "resume.pdf"]
```

**Output:**

```
["resume.pdf"]
```

**Constraints:**

- $1 \leq \text{files.length} \leq 10^5$
- File names are case-sensitive
- No repeated entries in output

**Hint:**

- Use *HashMap* to count occurrences.
- Collect names whose count exceeds 1.

**Method Signature:**

```
public List<String> findDuplicates(String[] files)
```

## Section 4 — Stack

### 16. Valid Parentheses

Given a string containing only '(', ')', '{', '}', '[' and ']', determine whether the string is valid. Every opening bracket must be closed by the correct type in the correct order.

Input:

```
"{[()]}"
```

Output:

```
true
```

Constraints:

- $1 \leq s.length() \leq 10^4$
- Only the six bracket characters
- Empty string is valid

Method Signature:

```
public boolean isValid(String s)
```

Hint:

- Push opening brackets; on a closing bracket, verify the stack top matches.
- The string is valid only if the stack is empty after full traversal.

### 17. Next Greater Element

For each element in an integer array, find the next element to its right that is strictly greater. If none exists, assign -1. Return results as an array of the same length.

Input:

```
[4, 5, 2, 10]
```

Output:

```
[5, 10, 10, -1]
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- Output must match input length

Method Signature:

```
public int[] nextGreaterElement(int[] nums)
```

Hint:

- Use a monotonic decreasing stack storing indices.
- When a larger element is found, pop the stack and resolve pending indices.

### 18. Remove Adjacent Duplicates

Given a string, repeatedly remove all adjacent duplicate characters until none remain. Apply the process in a single pass using a stack and return the final string.

Input:

```
"abbaca"
```

Output:

```
"ca" // abbaca -> aaca -> ca
```

**Constraints:**

- $1 \leq s.length() \leq 10^5$
- Lowercase English letters only
- Apply until no adjacent duplicates remain

**Method Signature:**

```
public String removeDuplicates(String s)
```

**Hint:**

- *Treat the stack top as the previous character; push or pop accordingly.*
- *The stack contents at the end form the result string.*

## 19. Decode String

Given an encoded string in the format  $k[\text{encoded\_string}]$ , decode it so that `encoded_string` is repeated exactly  $k$  times. Encodings may be nested.  $k$  is always a positive integer.

**Input:**

```
"3[a2[c]]"
```

**Output:**

```
"accaccacc"
```

**Constraints:**

- $1 \leq s.length() \leq 30$
- Digits, lowercase letters, and brackets only
- $k$  is in range  $[1, 300]$

**Method Signature:**

```
public String decodeString(String s)
```

**Hint:**

- *Use two stacks: one for repeat counts, one for string segments.*
- *On '[', push current state; on ']', pop and expand.*

## 20. Stock Span Problem

The stock span on a given day is the maximum number of consecutive days (including today) for which the price was less than or equal to today's price. Compute the span for each day.

**Input:**

```
[100, 80, 60, 70, 60, 75, 85]
```

**Output:**

```
[1, 1, 1, 2, 1, 4, 6]
```

**Constraints:**

- $1 \leq \text{prices.length} \leq 10^5$
- $1 \leq \text{prices}[i] \leq 10^5$
- Output must match input length

**Method Signature:**

```
public int[] stockSpan(int[] prices)
```

**Hint:**

- *Use a stack storing indices of days with unresolved spans.*
- *Pop indices whose prices are  $\leq$  current price;  $\text{span} = \text{index difference}$ .*

## Submission Guidelines

### Code

- Each solution must be a complete, compilable Java class.
- Use meaningful variable and method names.
- Follow Java naming conventions — camelCase for variables, PascalCase for classes.
- One problem per file. File name must match the class name.
- Include a main() method with at least the given example as a test case.

### GitHub

- Create a repository named java-collections-practice.
- Organise files into folders: arraylist/, hashset/, hashmap/, stack/.
- All solutions must be pushed before Monday morning.
- Commit messages must describe the problem — e.g., Add Problem 3 - Leaders in Array.
- Late pushes will not be considered during the review.

### README.md

- List all 20 problems with problem number and name.
- Include example input and output for each problem.
- Add at least one screenshot of a successful run per section.
- State the time and space complexity of each implementation.

---

All submitted code must be your own. You may reference documentation and study examples, but the final implementation must be written independently. Copied code will be identified during the Monday review.